1. (20 pts, 5 pts for each)

Mark by T(=true) or F(=false) each of the followings. You do not need to justify them.

a. Suppose problem P1 can be reduced to problem P2 in linear time (i.e.,P1 $\propto$O(n) P2). Then, if there exists a polynomial time algorithm for P1, there there exists a polynomial time algorithm for P2.

Ans : T

💔 Ans : F

Explanation: The direction of the reduction is key. If P1 reduces to P2, it means we can solve P1 using a solution to P2. So, if P2 has a polynomial-time algorithm, then P1 also does. But this statement says if P1 has a polynomial-time algorithm, then P2 does, which is not necessarily true. The reduction helps go from P2 $\longrightarrow$ P1, not the other way.

b. Quicksort (a typical version) is a stable sorting.

Ans : F

c. Hoare's partition has a better average-performance than Lomuto's partition in quicksort.

Ans : T

d. Select algorithm using median-of-medians rule runs in linear time with group of 5.

Ans : T

2. (25 pts, 5 pts for each)

Based on what we have discussed in class, give the best asymptotic running time for each of the problems (1) to (4) below, using the "big Oh" notation. It is assumed that T(1) = d for some constant d in all the recurrences. For Problem (5), please give the best number of comparisons. Just state the answers- you do not need to justify them.

(1) Finding the median in an unsorted set of size n, k $\leq$ n.

Ans: O(n)

(2) Quicksort on n element in worst case.

Ans: O(n^2)

(3) Finding the smallest element in a binary max heap with n keys.

Ans: O(log n)

💔 Ans : O(n)

In a binary max heap, the smallest element is not guaranteed to be at a specific position and is typically among the leaves. Since a binary heap with n keys has approximately n/2 leaves, finding the smallest element requires examining all the leaves, which takes O(n) time, not O(log n). Thus, the correct answer is O(n).

(4) Radix sort on an array of n integers in the range $[0, n^5 - 1]$ with base 10 representation.

Ans: O(n^5)

💔 Ans : O(n log n)

For Radix sort with base 10 representation, the time complexity depends on the number of digits (d) and the number of elements (n). The largest number in the range [0, n^5 - 1] is n^5 - 1, which has O(log n) digits in base 10 (since $\log_{10}(n^5) = 5 \log_{10} n$). Radix sort performs d passes, each taking O(n + b)

time, where b is the base (here, b = 10, a constant). Thus, the total time is O(d x n) = O(n log n), not $O(n^5)$. The correct answer is O(n log n).

(5) Given a set of n elements, a1, a2, . . . , an, find the maximum and minimum simultaneously, where the number of comparisons is counted. Ans: (3n)/2 −2, O(n)

3. (15 pts, 5 pts for each)

Given a set of n numbers, we wish to find the i largest elements in sorted order using a comparison-based algorithm. Find the algorithm that implements each of the following methods with the best asymptotic worst-case running time, and analyze the running times of the algorithms in terms of n and i.

(a) Sort the numbers, and list the i largest elements.

Ans: O(n log n) for sorting, then O(i) for listing the i largest elements. Total: O(n log n).

(b) Build a max-priority queue from the numbers, and call Extract-Max i items.

Ans: Building a max-priority queue takes O(n) time, and calling Extract-Max i times takes O(i log n) time. Total: O(n + i log n).

(c) Use an order-statistic algorithm to find the i largest number, partition around that number, and sort the i largest numbers.

Ans: The order-statistic algorithm (like Quickselect) runs in O(n) time to find the i-th largest number. Partitioning around that number takes O(n) time, and sorting the i largest numbers takes O(i log i). Total: O(n + i log i).

4. (20 pts)

Please answer each of the following problems shortly and concisely.

(1) (5 pts) Let B be a binomial heap of size 27. Please draw the shape of this binomial heap.

(2) (5 pts) Please use radix sort to sort the following integers: 129, 543, 638, 423, 732, 254, 147, 668, 333, 425 You need to show the steps.

(3) (5 pts) Given an n-element array A of real numbers, design an O(n) time algorithm which determines whether any value occurs more than n/7 times in A. Just write down your algorithm and the analysis for the time complexity.

Algorithm:

1. Count the occurrences of each number in A using a hash map.
2. For each number in the hash map, check if its count is greater than n/7.
3. If any number's count exceeds n/7, return true; otherwise, return false.

Time Complexity: O(n) for counting occurrences and O(k) for checking counts, where k is the number of distinct elements in A. Since $k \leq n$, the overall time complexity is O(n).

(4) (5 pts) Let A be an unsorted array of n numbers, and let B be an unsorted array of m numbers, where m is (possible much) smaller than n. A number may occur in A many times, but the numbers in B are distinct from each other. We are interested in finding out which element of B occurs most frequently in A. Describe an efficient algorithm for this problem. What is the time complexity of your algorithm?

Step 1 - Initialization:

Inserting m elements from B into the hash map takes O(m) time, as each insertion operation in a hash map is O(1) on average.

Step 2 - Counting:

Iterating through A requires O(n) time, as we process each of the n elements exactly once. For each element, checking if it exists in the hash map and incrementing its count are both O(1) average-time operations, assuming a well-implemented hash map with minimal collisions. Thus, this step totals O(n) time.

Step 3 - Finding the Maximum:

Scanning the hash map to find the key with the maximum value requires examining all m entries, which takes O(m) time.

Total Time Complexity:

Combining these steps: O(m) + O(n) + O(m) = O(n + m). Since the problem states that m is possibly much smaller than n, this is effectively O(n) in practice when m << n, but we express it as O(n + m) for generality.

5. (20 pts, 5 pts for each)

Let A_1, ... , A_5 be matrices with dimension 5 x 3, 3 x 4, 4 x 2, 2 x 3, 3 x 1, respectively. Let $C_{i,j}$ be the smallest number of scalar multiplications needed for computing the matrix product $A_i A_{i+1} ... A_j$, assuming that multiplying an r x s matrix and an s x t matrix take rst scalar multiplications. Mark by T(=True) or F(=False) each of the following statements:

(1) In the algorithm presented in class, $C_{3,5}$ is computed before $C_{1,4}$.

Ans: T

(2) $C_{2,4} = 24$.

Ans: F

(3) $C_{1,4}$ is derived from $C_{1,3}$ and $C_{4,4}$.

Ans: T

(4) The optimal order to multiply A_1, ... , A_5 with fewest number of scalar multiplications is $(A_1(A_2(A_3(A_4A_5))))$

Ans: F (but no) Ans : T

6. (10 pts, 5 pts for each)

Consider two teams, A and B, playing a series of games until one of the teams win n games. Assume that the probability of A winning a game is the same for each game and equal to p, and let the probability of A losing a game is q = 1 - p. (Hence, there are no ties.) Let P(i, j) be the probability of A winning the series if A needs i more games to win the series and B needs j more games to win the series.

(a) Set up a recurrence relation for P(i, j) that can be used by a dynamic programming algorithm.

P(i, j) means A already wins n-i and B already wins n-j games. The recurrence relation can be set up as follows:

P(i, j) = p x P(i - 1, j) + q x P(i, j - 1)

(b) Find the probability of team A winning a seven-game series if the probability of it winning a game is 0.4.